

## OpenMetadata con Apache Iceberg

L'obiettivo è stato quello di integrare OpenMetadata con l'ambiente Apache Iceberg già esistente. Il catalogo Iceberg era già presente in PostgreSQL e conteneva:

- *Catalog: default*
- *Namespace: test*
- *Table: prodotti*

L'obiettivo, pertanto, non è stato quello di creare un nuovo catalogo Iceberg, bensì di permettere a OpenMetadata di leggere e catalogare quello già esistente.

## Installazione

È stato installato OpenMetadata versione 1.12.10 tramite Docker, scaricando il file Docker Compose relativo alla versione utilizzata:

```
curl -sL -o docker-compose-postgres.yml https://github.com/open-metadata/OpenMetadata/releases/download/1.12.10-release/docker-compose-postgres.yml
```

L'ambiente risultante comprende diversi container:

- *openmetadata\_server*: contiene il server principale e la UI web accessibile sulla porta 8585
- *openmetadata\_postgresql*: è il database interno di OpenMetadata. A noi non servirà
- *openmetadata\_elasticsearch*: gestisce l'indicizzazione e la ricerca delle entità presenti nel catalogo.
- *openmetadata\_ingestion*: contiene Airflow e i connector Python utilizzati da OpenMetadata per collegarsi alle sorgenti e importarne i metadata.
- *execute\_migrate\_all*: è il container utilizzato durante l'installazione/aggiornamento per preparare lo schema interno di OpenMetadata.

## Collegamento del catalogo Iceberg

Successivamente, si è proceduto al collegamento del catalogo Iceberg già esistente con OpenMetadata. Il catalogo utilizzato originariamente era un SQL Catalog di PyIceberg. Nella configurazione del connector Iceberg, OpenMetadata proponeva diverse tipologie di catalogo, tra cui:

- *HiveCatalog*
- *RestCatalog*
- *GlueCatalog*
- *DynamoDBCatalog*

Non essendo disponibile una connessione diretta al SQL Catalog di PyIceberg, è stato scelto *RestCatalog*. Il RestCatalog è stato utilizzato come strato intermediario ed espone, tramite un API REST, il catalogo Iceberg memorizzato in PostgreSQL, senza dover creare un nuovo catalogo. A questo scopo è stata utilizzata l'immagine Docker :

```
apache/iceberg-rest-fixture:latest
```

Il container è stato configurato in modo da utilizzare:

- *JdbcCatalog*
- PostgreSQL: *pg\_db*
- database: *bdm\_prova*
- schema: *prova\_catalog*
- warehouse: *s3://lakehouse/*
- MinIO come endpoint S3.

Il comando utilizzato è stato:

```
docker run -d \
--name iceberg-rest \
--network openmetadata-docker_app_net \
-p 8181:8181 \
-v "$(pwd)/postgresql-42.7.5.jar:/usr/lib/iceberg-rest/postgresql-42.7.5.jar" \
-e CATALOG_WAREHOUSE=s3://lakehouse/ \
-e CATALOG_IO_IMPL=org.apache.iceberg.aws.s3.S3FileIO \
-e CATALOG_S3_ENDPOINT=http://minio:9000 \
-e CATALOG_S3_ACCESS_KEY_ID=admin \
-e CATALOG_S3_SECRET_ACCESS_KEY=<PASSWORD_MINIO> \
-e CATALOG_S3_PATH_STYLE_ACCESS=true \
-e AWS_REGION=us-east-1 \
-e CATALOG_CATALOG_IMPL=org.apache.iceberg.jdbc.JdbcCatalog \
-e CATALOG_CATALOG_NAME=default \
-e CATALOG_URI=jdbc:postgresql://pg_db:5432/bdm_prova?options=-c%20search_path%3Dprova_catalog' \
-e CATALOG_JDBC_USER=postgres \
-e CATALOG_JDBC_PASSWORD=<PASSWORD_POSTGRES> \
-e CATALOG_JDBC_SCHEMA_VERSION=VI \
apache/iceberg-rest-fixture:latest \
java -cp '/usr/lib/iceberg-rest/*:iceberg-rest-adapter.jar' \
org.apache.iceberg.rest.RESTCatalogServer
```

Il driver “postgresql-42.7.5.jar” è stato necessario affinché il server REST Java potesse collegarsi a PostgreSQL tramite JDBC.

Inoltre, OpenMetadata non poteva utilizzare localhost per comunicare con PostgreSQL, MinIO e Iceberg REST, poiché, all’interno di un container Docker, localhost identifica il container stesso. Per questo motivo è stata utilizzata la rete Docker “openmetadata-docker\_app\_net”. Alla stessa rete sono stati collegati i servizi interessati. In questo modo i container hanno potuto comunicare tra loro utilizzando direttamente i rispettivi nomi come hostname:

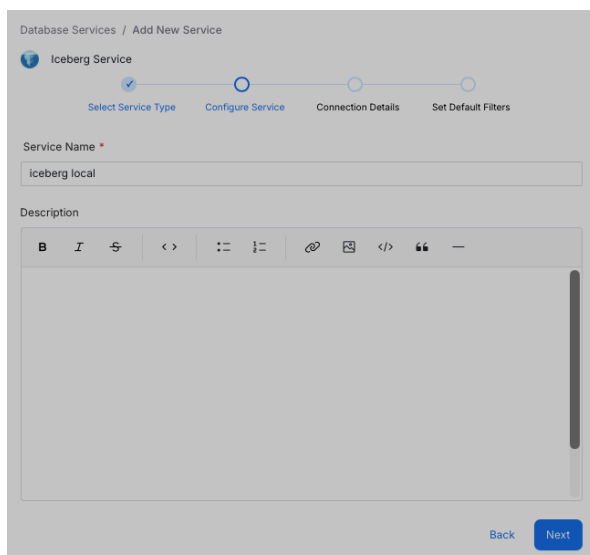
```
pg_db:5432
minio:9000
iceberg-rest:8181
ingestion:8080
```

## Configurazione di Iceberg in OpenMetadata

Dopodichè, si è proceduto alla configurazione della sorgente Iceberg tramite l’interfaccia grafica di OpenMetadata. Il percorso utilizzato è stato:

*Settings -> Services -> Databases -> Add New Service -> Iceberg*

I parametri di configurazione sono stati:



Database Services / Add New Service

**Iceberg Service**

☒ Select Service Type
 ☒ Configure Service
 ☐ Connection Details
 ☐ Set Default Filters

**IcebergCatalog**

Name \*

default

Connection \*

RestCatalogConnection

**RestCatalogConnection**

URI \*

http://iceberg-rest:8181

**AWS**

☐ Enable IAM Auth

AWS Access Key ID

admin

AWS Secret Access Key

\*\*\*\*\*

AWS Region \*

us-east-1

AWS Session Token

Endpoint URL

http://minio:9000

Profile Name

Role Arn For Assume Role

Role Session Name For Assume Role

OpenMetadataSession

Source Identity For Assume Role

Database Name

default

Warehouse Location

s3://lakehouse/

Ownership Property

owner

Una volta completata la configurazione e verificata correttamente la connessione, si è proceduto alla fase di metadata ingestion.

Il Metadata Agent si è collegato alla sorgente Iceberg tramite il REST Catalog, ha individuato le entità disponibili e ha importato i relativi metadata all'interno di OpenMetadata.

Database Services

**Iceberg local**   [Follow](#)

[Trigger AutoPilot](#) [0.1](#) 

[Domains](#) [Owners](#) [Tier](#)

--   --

[Insights](#) [Databases 1](#) [Agents 1](#) [Connection](#)

**Agents Status** [View more](#) 1 Successful

**Platform Insights** [View more](#)

Understand the metadata available in your service and keep track of the main KPIs coverage.

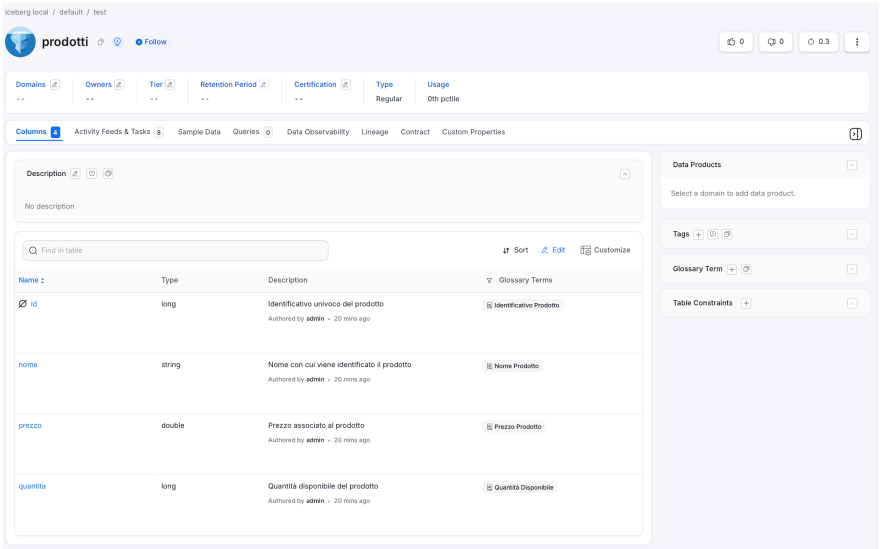
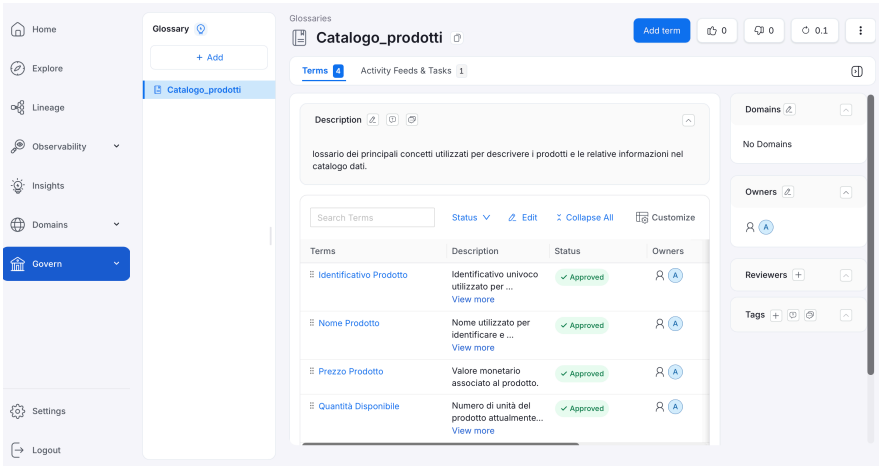
| Description Coverage  | PII Coverage          | Tier Coverage         | Ownership Coverage    | Healthy Data Assets   |
|-----------------------|-----------------------|-----------------------|-----------------------|-----------------------|
| 0%<br>In the last day | 0%<br>In the last day | 0%<br>In the last day | 0%<br>In the last day | 0%<br>In the last day |

**Total Data Assets**

- Database 1
- Database Schema 1
- Table 1

# Glossario

Successivamente è stato creato il Glossario tramite la sezione: *Governance -> Glossary -> Add Term*



# Lineage

Il lineage rappresenta le relazioni di provenienza e dipendenza tra i dati. Permette, ad esempio, di rappresentare una situazione nella quale una tabella viene utilizzata per generarne un'altra. Nell'ambiente attualmente configurato è presente solamente la tabella Iceberg *prodotti*. Non esistono quindi, al momento, relazioni significative tra più tabelle da rappresentare nel lineage.

